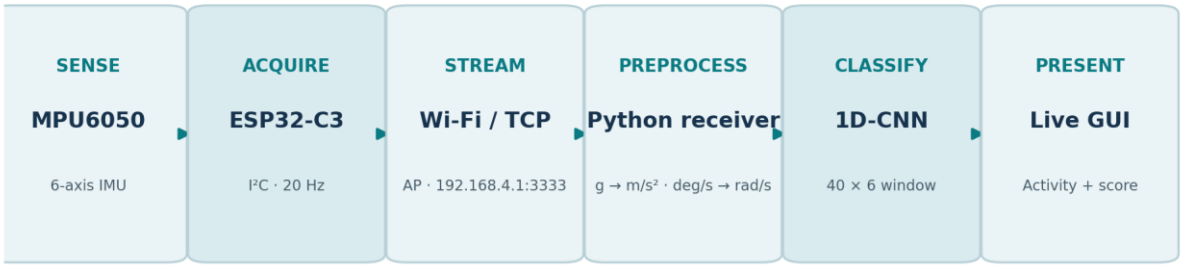
TECHNICAL PROJECT REPORT

Wearable Human Activity Recognition System

ESP32-C3 Super Mini • MPU6050 • Wi-Fi/TCP • 1D-CNN



Data contract: ESP32 transmits acceleration in g and angular velocity in deg/s; Python converts to dataset SI units.

System overview: sensing, Wi-Fi transmission, desktop preprocessing, and real-time classification.

SENSING	MODEL INPUT	OFFLINE EVALUATION
20 Hz	40 × 6	95.86%

Prepared from the supplied firmware specification, Python applications, training notebooks, and saved Keras model.

October 2026 | English Technical Documentation

Executive Summary

This report documents the design, data flow, model development, and practical operation of a wrist-worn human activity recognition (HAR) prototype. A six-axis MPU6050 measures motion; an ESP32-C3 Super Mini samples the sensor and streams measurements through its own Wi-Fi access point; a Windows laptop receives the stream through TCP, converts measurement units, segments the data into fixed-length time windows, and runs a seven-class one-dimensional convolutional neural network (1D-CNN). A Tkinter interface presents the predicted activity and softmax score. A separate Tkinter application collects labeled data for subsequent analysis.

The final, user-confirmed ESP32 stream uses acceleration in g and angular velocity in degrees per second (deg/s). The Python software converts these to m/s^2 and rad/s, respectively, to match the training dataset. This distinction is essential: a double conversion would alter the scale of the model inputs and invalidate the assumed preprocessing contract.

The supplied dataset-preparation notebook produces 19,551 windows, split into 15,640 training windows and 3,911 evaluation windows. The saved model accepts a (40, 6) tensor and predicts seven activities. The training notebook reports an evaluation accuracy of 95.86%. However, overlapping windows were created before a random window-level split, and the same evaluation subset was used for validation during training. Accordingly, the result describes that notebook evaluation and should not be interpreted as subject-independent field accuracy.

Scope of evidence. Implementation details and reported metrics are taken from the supplied files. No independent live-wearer accuracy, end-to-end timing benchmark, battery-life result, or network packet-loss measurement was provided; no such figures are asserted in this report.

Contents

1. Introduction • 2. Objectives and Scope

3. System Architecture • 4. Hardware Platform and Wiring

5. MPU6050 Configuration and Sampling • 6. ESP32-C3 Firmware Design

7. Wi-Fi Access Point and TCP Transport • 8. Data Contract and Unit Conversion

9. Dataset Source and Class Definitions • 10. Dataset Preparation and Windowing

11. Model Architecture • 12. Training Procedure and Reproducibility

13. Offline Evaluation and Discussion • 14. Real-Time Inference Application

15. Sliding-Window Schedule • 16. Personal Dataset Recorder

17. Installation and Operating Procedure • 18. System Testing and Verification

19. Limitations and Engineering Considerations • 20. Future Work

21. Conclusion • 22. References

Appendix A. Data Structures • Appendix B. Project File Guide

1. Introduction

Human activity recognition converts raw movement measurements into semantic descriptions of user behavior. Compared with camera-based approaches, inertial sensing is lightweight and does not require visual observation of the wearer. A wrist-mounted inertial measurement unit is attractive for monitoring movement patterns during daily tasks, exercise, and other physical activities.

Recognizing activities directly from an inertial time series remains challenging because sensor orientation, differences between users, movement intensity, and overlapping motion signatures can affect the observations. A learning-based classifier can discover discriminative temporal characteristics from labeled acceleration and angular-velocity signals. This prototype demonstrates an end-to-end implementation: measurement, wireless transport, data preparation, supervised model training, and desktop inference.

2. Objectives and Scope

The primary objective is to implement a wearable, real-time platform that identifies one of seven predefined activities from six-axis motion measurements. The project also provides a reproducible software path for training and a separate data-collection utility for personal recordings.

- Acquire three-axis acceleration and three-axis angular velocity from an MPU6050 at a nominal 20 Hz.
- Read the sensor using I^2C on an ESP32-C3 Super Mini and stream six numerical values over a local Wi-Fi/TCP connection.
- Prepare labeled sensor windows from the BandX-Activity dataset and train a seven-output 1D-CNN.
- Perform local desktop inference using the same feature order and unit conventions as the training data.
- Display current predictions and per-class scores in a graphical desktop interface.
- Collect new labeled sensor traces as CSV files for further experimentation.

Model inference runs on the laptop, not on the ESP32-C3. The device is a sensing and network-transmission node; on-device neural-network deployment is outside the present implementation.

3. System Architecture

The system comprises a wearable sensing node, a local wireless connection, and a desktop software layer. The MPU6050 measures motion, the ESP32-C3 reads consecutive registers over I^2C and acts as a TCP server, and one Python application connects as a client. The data receiver validates complete lines before buffering samples or recording them. The inference pipeline converts units, constructs a 40 × 6 tensor, and evaluates a saved Keras network.

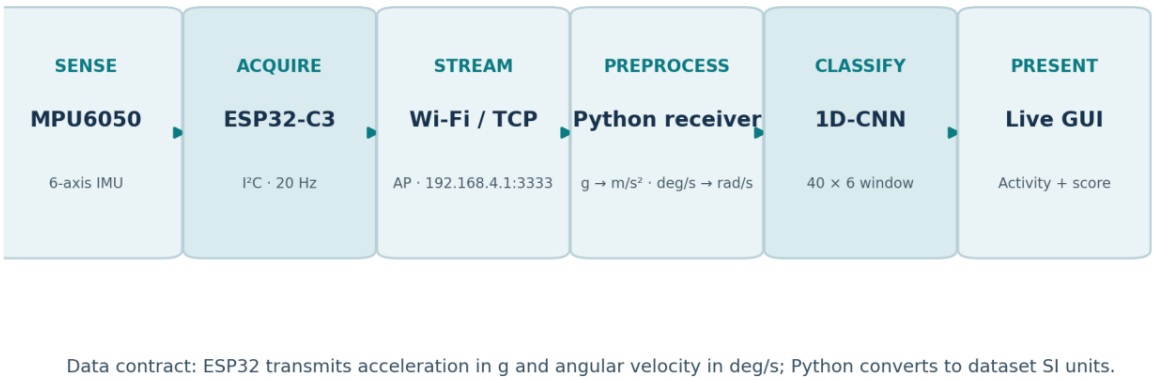


Figure 1. End-to-end operating path and required unit convention.

Subsystem	Role
MPU6050 IMU	Six-axis sensing: acceleration and angular velocity.
ESP32-C3 Super Mini	I^2C acquisition, sensor configuration, Wi-Fi access point and TCP server.
Python receiver	TCP byte-stream reassembly, parsing, validation, and buffering.
Preprocessing	Consistent axis order and conversion from g/deg/s to m/s^2/rad/s.
1D-CNN	Activity classification using fixed-duration sensor windows.
Tkinter applications	Live class display and separate labeled CSV recording.

4. Hardware Platform and Wiring

4.1. Major components

Component	Purpose / implementation
ESP32-C3 Super Mini	Compact ESP32-C3 microcontroller with integrated Wi-Fi.
MPU6050 module	Three-axis accelerometer and three-axis gyroscope.
Windows laptop	Wi-Fi TCP client, Keras model execution, graphical interface, and dataset storage.
USB cable / stable supply	Firmware programming and reliable power for the sensing node.
Optional CP2102 USB-to-UART adapter	Programming or serial diagnostics; not the operational sample-transport link.
Wristband mount / wiring	Secure positioning and electrical connections.

4.2. MPU6050-to-ESP32-C3 connections

MPU6050 terminal	ESP32-C3 Super Mini	Function
VCC	3.3 V	Sensor power
GND	GND	Shared reference
SDA	GPIO8	I^2C data
SCL	GPIO9	I^2C clock

The application uses GPIO8 for SDA and GPIO9 for SCL. These assignments are specific to the supplied ESP32-C3 Super Mini design and should not be replaced with pin mappings from a different ESP32 board. The firmware probes I^2C address 0x68 and, if necessary, 0x69. Stable power is important during Wi-Fi transmission; a limited-current USB-to-UART 3.3 V pin should not serve as the primary Wi-Fi power supply.

5. MPU6050 Configuration and Sampling

Setting	Configured value
I^2C clock	400 kHz
Accelerometer full scale	±2 g

Setting	Configured value
Gyroscope full scale	±250 deg/s
Digital low-pass filter	DLPF_CFG = 3
Sample-rate divider	SMPLRT_DIV = 49
Nominal output rate	20 samples/s
Output interval	50 ms
Measurement read	14 bytes starting at register 0x3B

The acquisition code reads a 14-byte block containing accelerometer outputs, temperature, and gyroscope outputs. The temperature word is not included in the six-element activity-recognition feature vector. At the selected sensor ranges, the configured raw sensitivities are 16,384 counts per g and 131 counts per degree per second. With the filter enabled, the approximate base rate is 1,000 Hz and the divider produces a nominal 20 Hz internal sample rate.

$$\textit{Sensor sample rate} = 1000 / (1 + 49) = 20\ \textit{Hz}$$

$$\textit{Nominal sample period} = 1 / 20 = 0.05\ \textit{s} = 50\ \textit{ms}$$

Sampling in firmware is scheduled with a micros()-based interval. As in many simple embedded streaming implementations, the nominal rate does not guarantee zero timing jitter under radio/network load or a blocked send operation. The supplied files do not contain an instrumented sampling-jitter analysis.

6. ESP32-C3 Firmware Design

The firmware performs a deterministic startup sequence: initialize the serial debug channel, open the I^2C bus, detect the MPU6050, configure its operating registers, create the Wi-Fi access point, and start a TCP server. During normal operation, it accepts a client, sends a CSV header once, then reads and transmits one six-axis measurement line per scheduled period.

1. Initialize I^2C with SDA = GPIO8, SCL = GPIO9, and a 400 kHz clock.
2. Probe addresses 0x68 and 0x69; stop with an error if the sensor is not detected.
3. Wake the MPU6050, configure the low-pass filter and ±2 g / ±250 deg/s ranges, and set the divider to 49.
4. Reset Wi-Fi mode, enable AP mode, configure IP/subnet, and use the supplied Super Mini transmit-power setting.
5. Start a TCP server on port 3333; accept one active streaming client.
6. Transmit an ASCII header, followed by comma-separated six-axis lines at a nominal 20 Hz.
7. Detect disconnection or transmission failure and return to client-accept state.

The embedded sampling routine extracts signed 16-bit register values and converts them to physical quantities before formatting the sample line. The final selected transmission contract is g for acceleration and deg/s for angular velocity; therefore, only raw-to-g and raw-to-deg/s conversion belongs in the transmitted values.

```
ax_g   = axRaw / 16384.0f;
ay_g   = ayRaw / 16384.0f;
az_g   = azRaw / 16384.0f;
gx_dps = gxRaw / 131.0f;
gy_dps = gyRaw / 131.0f;
gz_dps = gzRaw / 131.0f;
```

Important firmware reconciliation. The Arduino sketch originally pasted in the conversation applies

extra multipliers (9.80665 and π/180) and therefore actually transmits SI units. The final unit choice confirmed for this report is g and deg/s. The source sketch must use the six direct conversions above, or the Python input-unit selector must be adjusted accordingly. Never perform the same conversion twice.

7. Wi-Fi Access Point and TCP Transport

Parameter	Value
Wi-Fi mode	Access Point (AP)
SSID	ESP32-C3-MPU6050
Password	mpu6050wifi
Access point / server address	192.168.4.1
Gateway	192.168.4.1
Subnet	255.255.255.0
TCP port	3333
AP channel	6
Configured maximum AP stations	4
Active streaming clients	1 (replacement on new connection)
Transmit power API setting	WIFI_POWER_8_5dBm

The laptop joins the microcontroller-generated wireless network; no router, cloud service, or external internet connection is needed for the sensing and inference workflow. The application opens a TCP socket to 192.168.4.1:3333 after the access point becomes available. The firmware transmits a header and thereafter writes CRLF-terminated sample records.

Because TCP exposes a byte stream rather than guaranteed one-message-per-read boundaries, a receiver must accumulate bytes, split on line endings, and validate each complete record. Both supplied Python applications implement line-based parsing and ignore non-numeric headers. New connections replace an earlier active client; the recorder and the live GUI should therefore be used sequentially instead of simultaneously.

8. Data Contract and Unit Conversion

8.1. Final six-feature stream

```
ax,ay,az,gx,gy,gz
0.012,-0.006,1.008,0.150,-0.230,0.080
```

The sample shown is illustrative, not an experimentally recorded trace. The required sequence is [ax, ay, az, gx, gy, gz]. Acceleration is transmitted in g; gyroscope readings are transmitted in deg/s. Neither time nor activity label is included in the embedded TCP sample line.

Feature(s)	Physical quantity	Unit on ESP32 stream	Unit for training/inference

Feature(s)	Physical quantity	Unit on ESP32 stream	Unit for training/inference
ax, ay, az	Acceleration	g	m/s^2
gx, gy, gz	Angular velocity	deg/s	rad/s

8.2. Receiver-side conversion

$$a\ [m/s^2] = a\ [g] \times 9.80665$$

$$\omega\ [rad/s] = \omega\ [deg/s] \times \pi / 180$$

Both desktop applications provide an input-unit setting and default to "ESP32: g + deg/s". Under this setting, NumPy converts the incoming accelerations by multiplying by 9.80665 and angular velocities by π/180. The converted values match the unit convention used for model training and for saved personal CSV records. If a device already sends SI units, the GUI provides a "Dataset: m/s^2 + rad/s" option so the values pass through without conversion.

The training code does not fit an additional StandardScaler or other statistical normalization object. Introducing one in the live application without retraining would change the numerical input distribution. Axis order, mounting orientation, sample rate, time-window length, and class-index order are equally important for consistency.

9. Dataset Source and Class Definitions

The provided implementation uses the BandX-Activity human activity recognition dataset, associated with the Zenodo record 14543551. The project documentation describes this resource as containing MPU6050 recordings from 33 participants at a sampling rate of 20 Hz. The raw file schema contains a timestamp, an activity label, and six inertial measurements: time,label,ax,ay,az,gx,gy,gz.

Dataset label	Class index	English activity
C	0	Cycling
D	1	Downstairs
J	2	Jogging
T	3	Typing
U	4	Upstairs
Wa	5	Walking
Wr	6	Writing

The training notebook uses scikit-learn LabelEncoder, which produces the displayed alphabetical class order for the available labels. The live GUI hardcodes the equivalent English class-name sequence. This label mapping must be preserved when training a replacement model or modifying the graphical application.

10. Dataset Preparation and Windowing

Dataset_Creation.ipynb reads CSV files from a configured directory, checks the required eight-column schema, sorts each file by timestamp, and extracts the six numerical sensor channels. It then constructs fixed-length windows and retains a window only when every row carries the same activity label. Windows are

produced separately within each input file, protecting against accidental direct concatenation of unrelated file boundaries.

Preprocessing choice	Implementation
CSV required columns	time,label,ax,ay,az,gx,gy,gz
Nominal sampling rate	20 Hz
Training window length	2 s = 40 samples
Training overlap	50% = 20 samples
Training window stride	20 samples = 1 s
Window labels	Only homogeneous activity windows kept
Shuffle / split seed	random_state = 42
Window-level split	80% training / 20% evaluation
Saved data product	processed_dataset.npz

Feature matrices are stored in arrays of shape (number_of_windows, 40, 6). No timestamp or text label enters the neural network as an input feature. The output NPZ contains x_train, x_test, y_train, and y_test.

Set	Windows	Per-window shape
Training	15,640	40 × 6
Evaluation	3,911	40 × 6
Combined	19,551	40 × 6

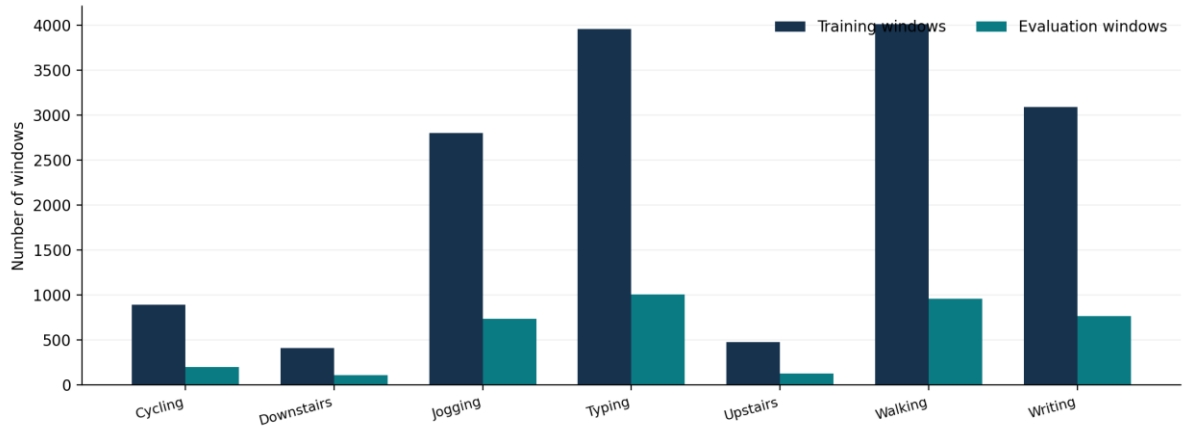


Figure 2. Class distribution printed by the dataset-preparation notebook.

The class distribution is uneven. In particular, Downstairs and Upstairs have substantially fewer windows than the frequently represented Typing and Walking classes, which may contribute to differences in class-wise recall.

11. Model Architecture

The prediction model is a one-dimensional convolutional neural network implemented as a Keras Sequential model. Temporal convolution filters extract motion patterns from each window, and dense layers combine learned features into class scores. Batch normalization and dropout are used in the network, while max pooling reduces temporal resolution.

Order	Layer	Configuration
1	Conv1D	64 filters, kernel size 3, ReLU
2	BatchNormalization	After first convolution
3	MaxPooling1D	Pool size 2
4	Dropout	Rate 0.30
5	Conv1D	128 filters, kernel size 3, ReLU
6	BatchNormalization	After second convolution
7	MaxPooling1D	Pool size 2
8	Dropout	Rate 0.30
9	Flatten	Flatten temporal features
10	Dense	128 units, ReLU
11	Dropout	Rate 0.50
12	Dense	7 units, softmax

The input tensor is (batch_size, 40, 6), and the output is a seven-element softmax vector for each window. The supplied trained_model.h5 file was independently inspected for layer configuration; it contains the specified input dimensions and final seven-unit dense layer. The live Python program checks for an input shape equivalent to (None, 40, 6) before activating predictions.

12. Training Procedure and Reproducibility

train_model.ipynb loads processed_dataset.npz, converts sensor tensors to float32, encodes categorical labels with LabelEncoder, and transforms label indices into one-hot vectors. The model is compiled with Adam optimization, categorical cross-entropy loss, and accuracy monitoring.

Training variable	Setting
Optimizer	Adam
Loss	categorical_crossentropy
Training epochs	50
Batch size	32
Numeric data type	float32
Label encoding	LabelEncoder + one-hot
Validation data	x_test and y_test (the notebook evaluation split)
Saved model	trained_model.h5
Runtime framework	TensorFlow / Keras

For re-execution, the CSV source directory should first be configured in Dataset_Creation.ipynb. After generating the NPZ, train_model.ipynb is executed using the NPZ in its working directory; its final Keras model is saved as trained_model.h5. The training notebook does not persist a separate fitted scaler. Although LabelEncoder is used during training, the inference GUI uses the corresponding fixed class-name ordering instead of loading a separate serialized encoder.

The provided training procedure does not enforce a subject-disjoint partition, does not create a separately untouched holdout test set, and does not document multiple randomized training runs. Additional reproducibility work should preserve package versions, random seeds across the full stack, hardware and data provenance, and the exact dataset release.

13. Offline Evaluation and Discussion

13.1. Reported accuracy and per-class performance

The train_model.ipynb output explicitly reports 95.86% evaluation accuracy on 3,911 windows. The corresponding scikit-learn classification report prints the precision, recall, F1-score, and support values below. These figures are copied from the supplied recorded notebook output, not from a new training run.

Activity	Precision	Recall	F1	Support
Cycling	0.99	1.00	0.99	201
Downstairs	0.96	0.66	0.78	112
Jogging	0.98	0.96	0.97	734
Typing	0.98	0.98	0.98	1,010
Upstairs	0.89	0.56	0.69	128
Walking	0.91	0.99	0.95	961
Writing	0.97	0.99	0.98	765
Macro avg.	0.95	0.88	0.91	3,911
Weighted avg.	0.96	0.96	0.96	3,911

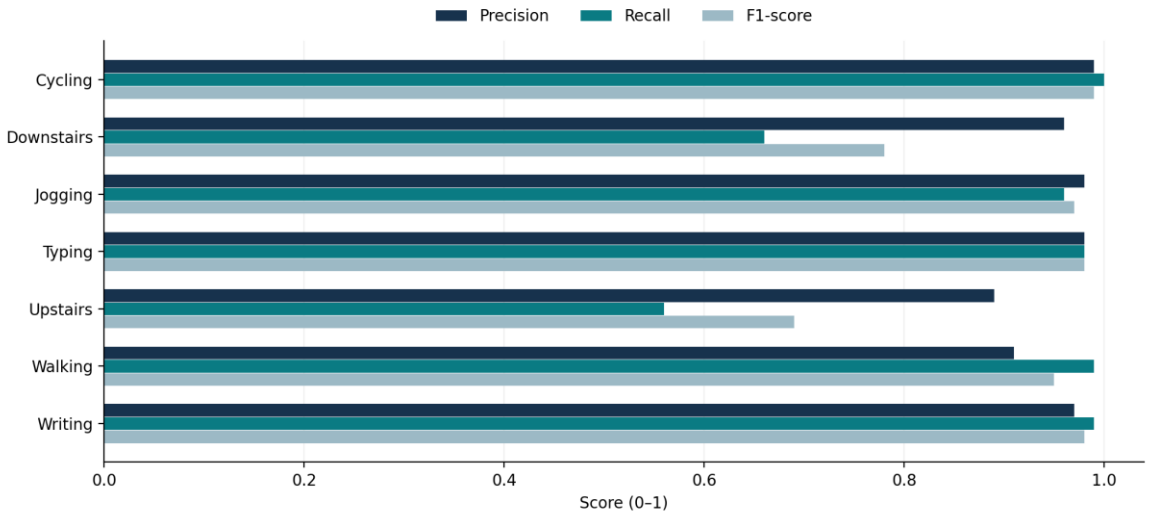


Figure 3. Class-wise precision, recall, and F1-score from the stored notebook report.

13.2. Interpretation

The highest recall is reported for Cycling (1.00), while Walking and Writing both reach 0.99. In contrast, Downstairs recall is 0.66 and Upstairs recall is 0.56. Consequently, the weighted accuracy alone masks meaningful differences between categories. The lower sample counts for stair-related activities are one plausible contributing factor, although the supplied evaluation does not establish a causal explanation.

13.3. Critical evaluation-design caveat

Window overlap and data leakage risk. Training windows are generated with 50% overlap before a random window-level train/evaluation partition. Highly similar, overlapping time windows, and potentially segments from the same subjects, can therefore appear in both subsets. Moreover, the evaluation subset is passed to model.fit as validation_data and subsequently used for final metrics. The 95.86% result is an offline development-split metric, not an independent generalization estimate for unseen users.

A stronger evaluation protocol would split data by participant before window creation, reserve a held-out test group that is not used during model selection, report repeated-run variability, and measure accuracy under actual wrist-mounted operation.

14. Real-Time Inference Application

HAR_Zenodo_WiFi_Live_GUI.py provides the desktop prediction interface. It uses Tkinter for controls, Python sockets for Wi-Fi/TCP communication, a background receiving thread to avoid blocking the graphical interface, a queue for transferring received lines to the main UI loop, NumPy for numerical arrays, and TensorFlow/Keras for model loading and inference.

8. Select the saved trained_model.h5; the application checks that it accepts 40 samples and six channels.
9. Join the ESP32-C3 access point and open a TCP connection to 192.168.4.1:3333.
10. Read complete newline-terminated records in a background thread and discard non-numeric headers or malformed lines.
11. Convert g and deg/s values to m/s^2 and rad/s in float32.
12. Append valid samples to a rolling deque of capacity 40.
13. When an inference window is due, create a (1, 40, 6) tensor and call model.predict.
14. Choose the class with the highest returned score; update the predicted label, confidence display, sample count, prediction count, and per-class table.

The displayed “confidence” is the highest softmax output value multiplied by 100. Such scores represent the network output and should not automatically be treated as calibrated statistical probabilities for a new wearer or environment. No independent confidence-calibration analysis is included in the supplied materials.

15. Sliding-Window Schedule

Both training and live inference feed 40-sample windows to the network, but their overlap policies differ. Training uses a 20-sample stride (20-sample overlap). The live application uses a 30-sample stride (10-sample overlap), retaining ten measurements from the previous window.

Parameter	Training dataset	Live inference
Window length	40 samples	40 samples
Window duration at 20 Hz	2 s	2 s
Overlap	20 samples	10 samples
Stride	20 samples	30 samples
Nominal new-window interval	1.0 s	1.5 s

```
Live window 1:  samples   0–39
Live window 2:  samples  30–69
```

```
Live window 3:  samples  60–99
```

Live prediction step = (40 − 10) / 20 Hz = 1.5 s

The first prediction is possible only after a complete initial 40-sample window has arrived, corresponding to about 2 seconds of nominal sensor data. Thereafter, the software attempts a new prediction every 30 additional samples. This sampling-derived cadence must not be confused with a directly measured communication delay or processing latency.

16. Personal Dataset Recorder

HAR_Zenodo_WiFi_Dataset_Recorder.py is a separate Tkinter tool that captures labeled measurements from the same Wi-Fi stream. Its user interface includes address/port controls, unit-selection options, an activity-label selector, a CSV file path, Start/Stop buttons, and live information about samples written and recording duration.

Each complete TCP sample is assigned a timestamp on the laptop as close as possible to line receipt, expressed in Unix epoch milliseconds. The program converts incoming g/deg/s values to SI units, then writes a row with time, label, ax, ay, az, gx, gy, and gz. An existing CSV is opened in append mode, and a header is written only for a new empty file.

```
time,label,ax,ay,az,gx,gy,gz
<Unix-ms>,Wa,<ax_m/s²>,<ay_m/s²>,<az_m/s²>,<gx_rad/s>,<gy_rad/s>,<gz_rad/s>
```

The timestamps are host-reception times rather than acquisition timestamps recorded by the ESP32-C3. Consequently, they may reflect Wi-Fi or operating-system scheduling variation. For controlled experiments, sessions should be kept distinct, labels checked before recording, and data from disconnections and transitions reviewed before training.

17. Installation and Operating Procedure

17.1. Required software

A Windows computer with a compatible Python 3 environment is used for the supplied desktop software. Core packages include NumPy and TensorFlow for inference, plus pandas, scikit-learn, matplotlib, and Jupyter for dataset processing and training. Tkinter, sockets, queues, and threading are standard Python modules, though Tkinter availability depends on the Python distribution.

```
py -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install numpy tensorflow pandas scikit-learn matplotlib jupyter
```

17.2. Deploy the sensing firmware

15. Wire the MPU6050 to 3.3 V, ground, GPIO8/SDA, and GPIO9/SCL.
16. Use the ESP32 board-support package and select the appropriate ESP32-C3 target in Arduino IDE.
17. Compile and upload the g/deg/s-output Wi-Fi firmware configuration.
18. Check serial diagnostics for successful MPU6050 discovery and access-point startup.
19. Connect the laptop to SSID ESP32-C3-MPU6050 using the configured password.

17.3. Start the live classifier

```
python HAR_Zenodo_WiFi_Live_GUI.py
```

20. Select trained_model.h5 (or use the automatically populated path if the model is beside the script).
21. Verify host 192.168.4.1 and port 3333.
22. Keep the input selection at “ESP32: g + deg/s”.
23. Load the model and click Connect.
24. Secure the wearable, perform a supported activity, and inspect the displayed label and class scores.
25. Disconnect before running the separate dataset recorder.

17.4. Record additional labeled data

```
python HAR_Zenodo_WiFi_Dataset_Recorder.py
```

Choose a label from C, D, J, T, U, Wa, and Wr; select the CSV output path; verify g/deg/s input mode; then start and stop recording each labeled segment. After recording, inspect the data structure and verify that the values are already in SI units before processing the CSV with the dataset notebook.

17.5. Rebuild the model

Download the dataset from the cited source, set the local CSV folder in Dataset_Creation.ipynb, execute the notebook to generate processed_dataset.npz, then run train_model.ipynb in an environment where it can read the NPZ. Reload the resulting trained_model.h5 in the live GUI. Class mapping, tensor dimensions, and the physical measurement convention must remain aligned.

18. System Testing and Verification

The provided files establish the implemented software pathways but do not include a full bench-test log of the assembled wristband. The following procedure is a reproducible verification plan for checking the final system without implying that all listed measurements have already been performed.

Test	Expected observation / check
I^2C detection	Sensor responds at 0x68 or 0x69 and configuration succeeds.
Access point	SSID is visible and laptop joins local network.
TCP connection	Client reaches 192.168.4.1:3333 and receives a header followed by six-value lines.
Unit sanity check	With a suitable static orientation, one accelerometer axis is near ±1 g; stationary gyro readings are near 0 deg/s apart from bias.
Data validation	Header/malformed lines are ignored without entering model input.
Window formation	First inference after 40 valid samples, then one attempt per 30 additional samples.
Inference dimensions	Model accepts one (1,40,6) window and returns seven class scores.
Dataset recording	CSV includes Unix-ms timestamp, selected label, and six converted SI values.
Recovery	Disconnection is handled; reconnecting resets the rolling prediction buffer.

For a quantitative real-world study, log ground-truth activity labels, per-class outcomes, wall-clock timestamps, session/subject identifiers, radio connection events, and battery current under representative use. Such data would make it possible to estimate live accuracy, latency, timing jitter, and power demand instead of inferring them from nominal settings.

19. Limitations and Engineering Considerations

- Evaluation leakage risk: overlapped windows were split randomly, rather than separated by subject or acquisition session.
- Validation/test reuse: the same evaluation subset was used during model.fit and final evaluation.
- Class imbalance and uneven recall: stair-related activities exhibit lower recall than several other classes.
- Wrist placement and orientation: the model is sensitive to feature-axis changes and differences from the training attachment configuration.
- Single active TCP client: a new connection replaces an existing streaming client.
- No device-origin timestamps: the recorder timestamps on receipt at the laptop, not at acquisition.
- No field-system metrics: live classification accuracy, delay, sampling jitter, packet loss, and battery lifetime remain unmeasured in the provided files.
- Laptop-dependent prediction: an ESP32-C3 alone cannot run this saved Keras desktop model as implemented.
- Security and deployment: the access-point password is fixed in the example firmware and should be changed if deployed beyond a controlled prototype setting.

20. Future Work

The following development steps would strengthen both the scientific validity and practical utility of the system:

26. Create subject-independent training, validation, and held-out testing partitions before generating overlapping windows.
27. Collect additional traces for Upstairs and Downstairs and assess class-specific confusion patterns.
28. Record data across multiple participants, wrist positions, and movement intensities to quantify domain shift.
29. Measure true acquisition timestamps, lost records, network latency, jitter, and sample-rate stability.
30. Add hardware timestamping, per-session metadata, and explicit device/sensor calibration procedures.
31. Evaluate prediction smoothing, confidence calibration, and rejection of uncertain classifications.
32. Measure current consumption and design an integrated rechargeable wristband power subsystem.
33. Explore model compression or an alternative on-device inference platform for standalone operation.

21. Conclusion

The implemented prototype integrates six-axis sensing, local Wi-Fi TCP streaming, Python data processing, and deep-learning classification into a coherent wearable HAR workflow. Its ESP32-C3 Super Mini samples an MPU6050 at a nominal 20 Hz and transmits acceleration in g and angular velocity in deg/s. Python converts these values to the training convention, builds 40 × 6 temporal windows, and executes a saved seven-class 1D-CNN. Two desktop applications support immediate classification and labeled data recording.

The archived notebook output reports 95.86% accuracy on its 3,911-window evaluation split. This is a useful baseline for the provided software pipeline, while the overlapping-window split and validation reuse impose important limitations on generalization claims. The next stage should focus on controlled live-wearer testing and subject-independent evaluation, followed by robust wearable packaging and power optimization.

22. References

[1] B. Saha and R. Samanta, “BandX-Activity: Human Activity Recognition Dataset with Demographics Using the MPU6050 Sensor,” Zenodo, 2024. doi: 10.5281/zenodo.14543551.
https://zenodo.org/records/14543551

[2] Dataset_Creation.ipynb, project-supplied dataset creation and windowing notebook.

[3] train_model.ipynb, project-supplied 1D-CNN training and recorded classification evaluation notebook.

[4] trained_model.h5, supplied saved Keras model; inspected architecture corresponds to a 40 × 6 input and seven output classes.

[5] HAR_Zenodo_WiFi_Live_GUI.py, project-supplied Wi-Fi TCP live-inference application.

[6] HAR_Zenodo_WiFi_Dataset_Recorder.py, project-supplied Wi-Fi TCP labeled-recording application.

[7] ESP32-C3 / MPU6050 Arduino source provided in the project conversation, interpreted together with the user-confirmed final g/deg/s transmission contract.

Appendix A. Data Structures and Constants

A.1. Embedded six-value CSV record

```
ax,ay,az,gx,gy,gz
<g>,<g>,<g>,<deg/s>,<deg/s>,<deg/s>
```

A.2. Personal recording CSV

```
time,label,ax,ay,az,gx,gy,gz
<epoch_ms>,<class>,<m/s²>,<m/s²>,<m/s²>,<rad/s>,<rad/s>,<rad/s>
```

A.3. Core model and acquisition constants

Sampling: 20 Hz; input window: 40 samples × 6 features; live overlap: 10 samples; inference stride: 30 samples; activity classes: 7; TCP host: 192.168.4.1; TCP port: 3333.

Appendix B. Project File Guide

Artifact	Function
Dataset_Creation.ipynb	Read source CSVs, create windows, split, save processed_dataset.npz.
train_model.ipynb	Build/train CNN, produce classification metrics, save model.
trained_model.h5	Saved network artifact loaded by live GUI.
HAR_Zenodo_WiFi_Live_GUI.py	Connect to TCP server and display real-time predictions.
HAR_Zenodo_WiFi_Dataset_Recorder.py	Record labeled TCP measurements to a dataset-compatible CSV.
README.md	GitHub-oriented technical overview and setup instructions.
ESP32-C3 firmware sketch	Configure sensor, host AP/TCP endpoint, and transmit g/deg/s data.